Assignment 2 Report

Liam Watson

May 23, 2026

1 Introduction

This report details answers to key questions posed throughout the Digital Systems Project Assignment and analyses all waveforms given by each module using a Testbench. We aim to highlight key findings and confirm all output behaves as expected after testing.

2 Report 1

Report 1

When a module needs to be extended mid-design, several approaches are possible: modify the original in place; copy it, modify the copy, and leave the original untouched; or copy it, modify the copy, and rewrite the original as a thin wrapper around the new module. Can you think of any other options? For this specific case, which approach would you take, and how would you name the resulting module or modules? Justify your choices and discuss their trade-offs relative to the alternatives.

Figure 1: Screenshot taken from Digital_Watch_Part_2.pdf

In this instance we would prefer to create a new version of this module with a restart feature called **up_down_counter_rst**.

3 Report 2

Report 2

Do the assertions above fully capture the behaviour of an up-down counter with reset? If not, identify any behaviours left unspecified, or any properties a reasonable person might expect that the assertions do not guarantee.

Figure 2: Screenshot taken from Digital_Watch_Part_2.pdf

At this time it appears that verification may not have captured the ability to manually increment up and down nor if we wish to make active changes to **MAX** or if we wish to change the mininum value for which the counter start. Given our use case these features are not currently intended for our design.

4 Report 3

Report 3

The configuration file verifies the counter for three specific values of `MAX`. Since this does not cover all possible values, how would you formally prove correctness when this module is used in a larger design?

Figure 3: Screenshot taken from Digital_Watch_Part_2.pdf

When used in a larger design we can use proof by induction which will allow us to say with high confidence that our counter will scale for for higher values of **MAX**.

5 Report 4

Report 4

Include the counterexample waveform. The general reason for this failure was explained above; use the waveform to identify the specific counterexample the checker found and explain why it is one. Then explain why restoring the assertion allows the induction step to succeed.

Figure 4: Screenshot taken from Digital_Watch_Part_2.pdf

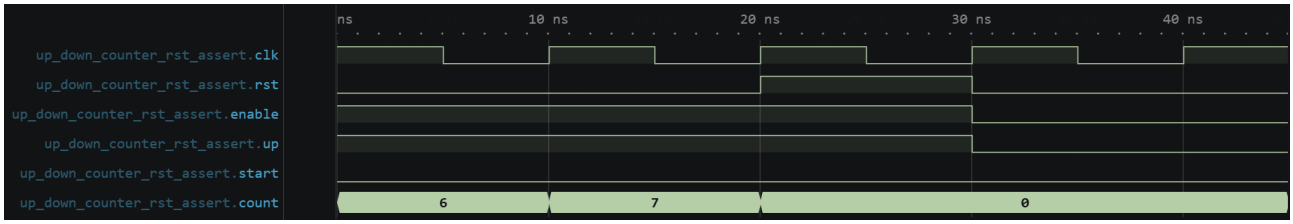


Figure 5: Measured waveform from up_down_counter_rst_assert

The induction check initialises count to 6. This fails as count then wraps to 0 from a value higher than MAX (4). Restoring the assertion then ensures that MAX can never exceed MAX meaning count can never be set above the value listed in the waveform above.

6 Report 5

Report 5

Before reading further, sketch how you would approach the design.

Figure 6: Screenshot taken from Digital_Watch_Part_2.pdf

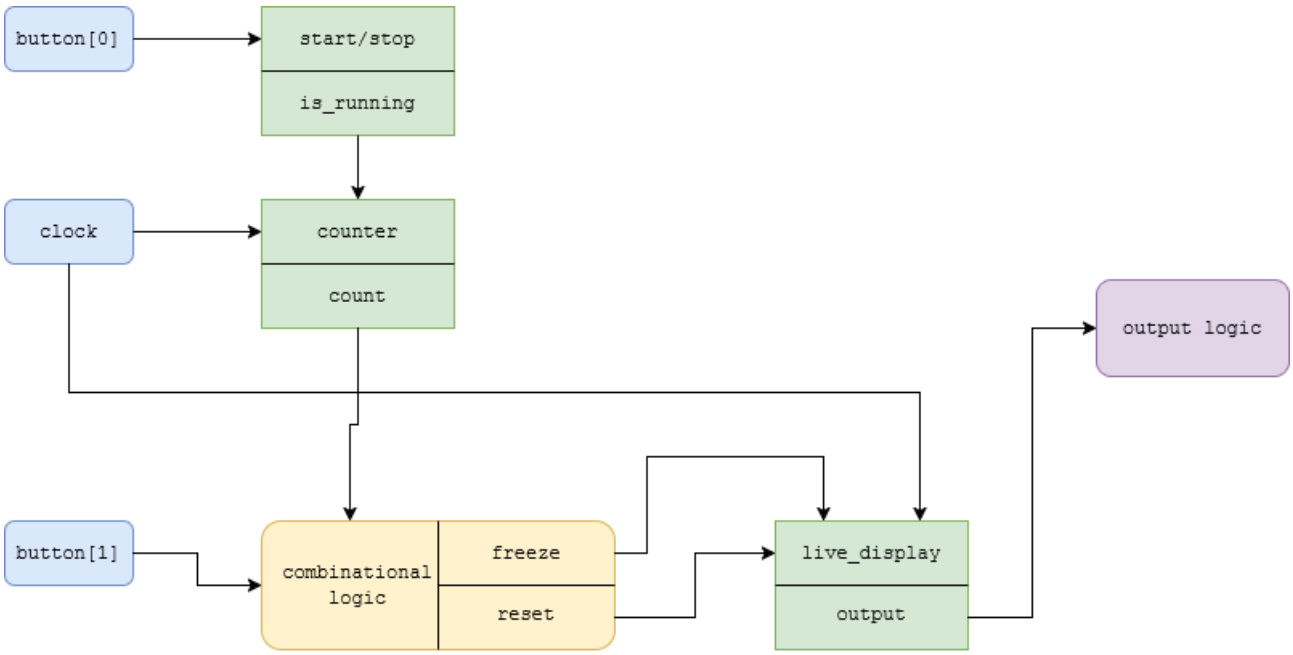


Figure 7: Sketch from diagram.io

7 Report 6

Report 6

Using a feature layering approach, how would you decompose the stopwatch into layers? Describe the responsibility of each layer.

Figure 8: Screenshot taken from Digital_Watch_Part_2.pdf

I would decompose the stopwatch into several layers. First we would handle our counter module and pass button[0] as an enable to control if the counter is running. I would then create a layer for the live display. I would then use combinational logic to determine the inputs to control the state for live display and use output logic to drive the stopwatch display.

8 Report 7

Report 7

Draw a block diagram of the design described above, clearly showing the data path and control signals. Compare it with your initial sketch from Section 4.2 and comment on any differences.

Figure 9: Screenshot taken from Digital_Watch_Part_2.pdf

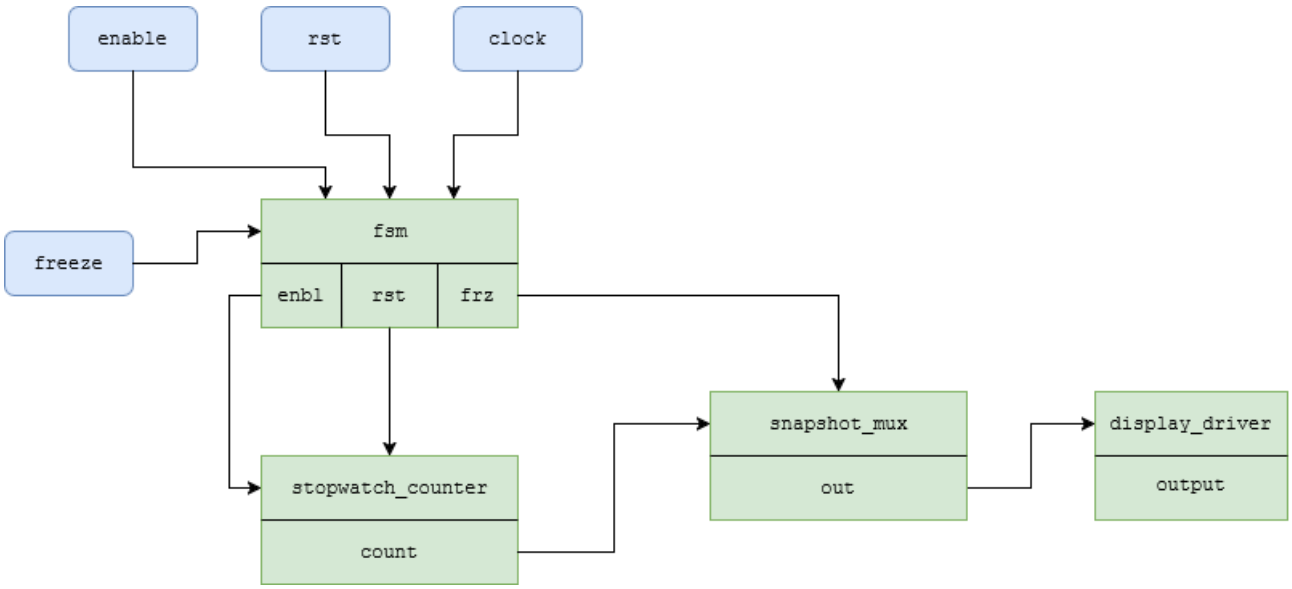


Figure 10: Block diagram created using diagram.io

9 Report 8

Report 8

Explain how your implementation satisfies the first-tick timing constraint: the first centisecond increment must occur one centisecond after `enable` goes high, not sooner.

Figure 11: Screenshot taken from Digital_Watch_Part_2.pdf

Using the **restartable_rate_generator** we generate a tick after one centisecond. This is then used alongside our enable signal to drive the **cascade_counter** module input.

10 Report 9

Report 9

Draw a diagram showing how this is implemented in hardware.

Figure 12: Screenshot taken from Digital_Watch_Part_2.pdf

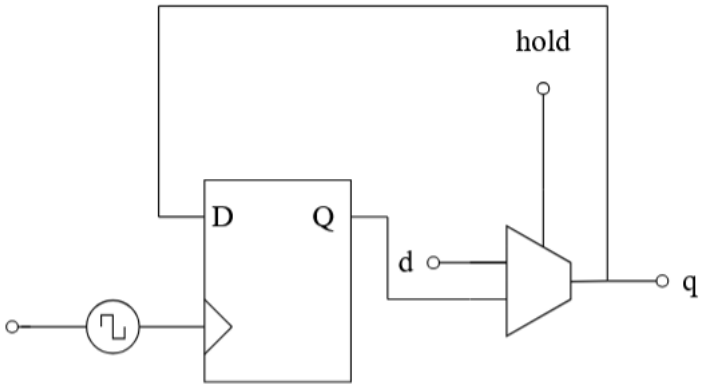


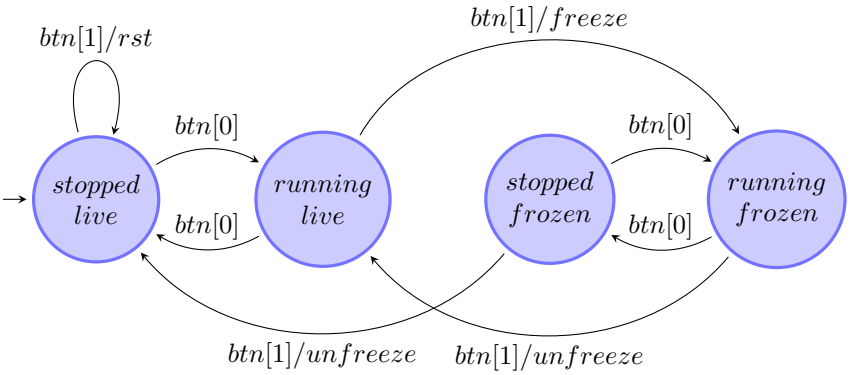
Figure 13: Created at tikzmaker.com

11 Report 10

Report 10

Draw the FSM state diagram before implementing it. After completing the implementation, note any differences between your diagram and the final design, and reflect on what caused them.

Figure 14: Screenshot taken from Digital_Watch_Part_2.pdf



12 Report 11

Report 11

Find the datasheet for the 74LS193 synchronous 4-bit up/down counter. Using the schematic diagram in the datasheet, explain how the borrow output is generated and why it behaves as it does.

Figure 15: Screenshot taken from Digital_Watch_Part_2.pdf

The borrow output is used to cascade the up and down counting functions. The borrow output produces a pulse equal in width to the count down input when the counter underflows. This allows the counter to be easily

cascaded by feeding the borrow ouput into the count down or up inputs.

In the schematic diagram the BORROW OUTPUT comes from a 5 input NAND gate. Its first input is connected to an inverted input DOWN COUNT, and inverted outputs from Q_A, Q_B, Q_C, and Q_D. This means borrow out remains high until all inputs and COUNT DOWN are low for half negedge clock cycle.

13 Report 12

Report 12

Describe your integration plan: which features did you implement first, and why?

Figure 16: Screenshot taken from Digital_Watch_Part_2.pdf

The first feature implemented was the set mode feature. This felt the most natural and served as a basis with which to test the countdown timer features once are able to set the timer. After this was implemented Stopped Mode

14 Report 13

Report 13

The timer requires an FSM to manage its three modes. Explain your FSM design: what are the states, what are the transitions, and how did you encode the states? Justify your choices.

Figure 17: Screenshot taken from Digital_Watch_Part_2.pdf

I used a simply FSM to with to determine if the counter is running. Using a combinational block we determine whether to toggle the state between running or not running and provide a guard to prevent the state from entering into running if in edit mode or if the count is not above zero.

15 Report 14

Report 14

Looking back across both assignments, reflect on your experience designing and implementing a digital watch on an FPGA. What went well, what was harder than expected, and what would you do differently if you were starting over?

Figure 18: Screenshot taken from Digital_Watch_Part_2.pdf

Overall this project presented a great learning experience and felt like working in a professional environment. The project ran fairly smoothly for me in terms of setting up the project and using the tests to ensure each module was working correctly. Certain modules I found difficult to implement even with the guidance the assignment. I think I would take a little more time to plan each module and understand each step exactly prior to implementation, and attempt to understand the validation test more thoroughly to assist with debugging.

16 Report 15

Report 15

Estimate how much time you saved compared to having used only Quartus. Break down the time savings between better editing features, linting, and automated tests plus formal verification. Without these tools, at what stage would you have discovered a problem, and how much harder would it have been to locate and fix the source of the error?

Figure 19: Screenshot taken from Digital_Watch_Part_2.pdf

Given Quartus's compilation times alone I would estimate several hours have been saved, let alone the formal verification tools effectiveness in debugging the code. The time spent learning these new tools was paid back ten-fold. Without these tools some of the bugs may have caught using testbenches however as the project grew and without formal verification tools, it would have become increasingly difficult to track down where the problems were and which module was causing them.

17 Report 16

Report 16

Include `elaborated.png` in your report. Yosys has elaborated the design but has not yet synthesised anything. The diagram contains five numbered elements ($1 to $5). Two are labelled `PROC`: one cites line numbers from `rtl/mod_n_counter.sv` and corresponds to the `always_ff` block; the other, at line 0.0–0.0, represents the initialisation of `count`. The remaining three cells are $lt, $add, and $mux. What does each compute? Explain how the three work together to implement the counter's next-state logic.

Figure 20: Screenshot taken from Digital_Watch_Part_2.pdf

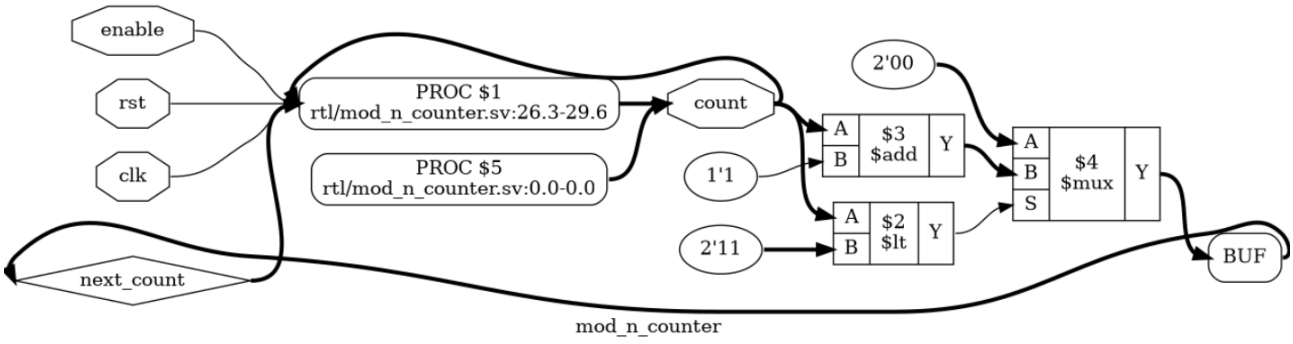


Figure 21: elaborated.png generated for mod_n_counter.sv using Yosys

$add works by adding 1'b1 to the count value and $lt is a comparison operator which checks to see if count is less than MaxCount (2'11). These are fed in to $mux where A is '0 and B is count + 1'b1. The output of $lt is then used as the select signal to determine the output of the $mux. These work together to drive our next state logic.

18 Report 17

Report 17

Include `proc.png` in your report. The two `PROC` cells from `elaborated.png` have been replaced by a single cell labelled `$sdffe`. What do you think `$sdffe` stands for? It has four ports: `CLK`, `D`, `EN`, and `SRST`; what does each correspond to in the original `always_ff` block? The combinational cells $lt, $add, and $mux are unchanged. What does this tell you about what `proc` and `opt` did and did not change?

Figure 22: Screenshot taken from Digital_Watch_Part_2.pdf

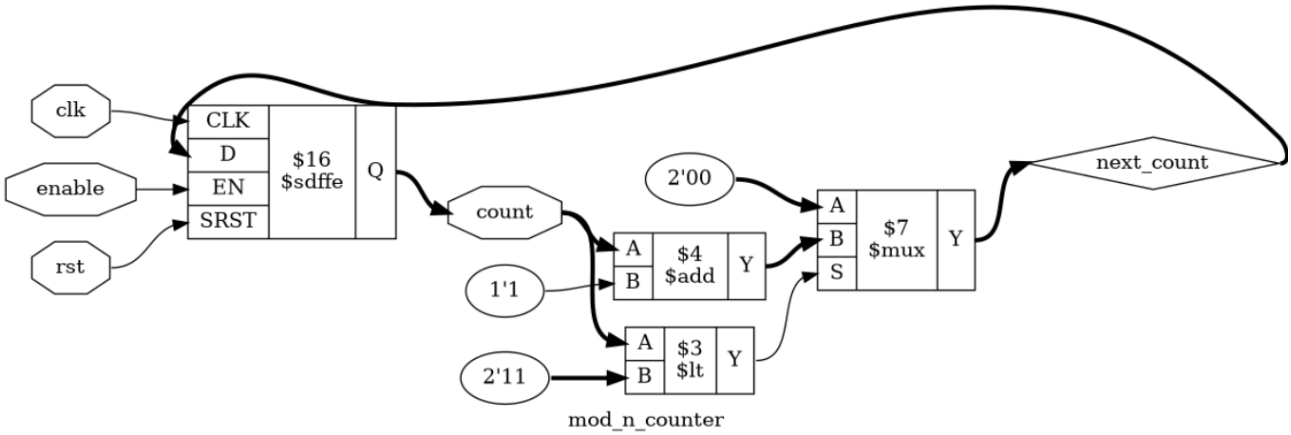


Figure 23: proc.png generated for mod_n_counter.sv using Yosys

$sdffe stands for synchronous d flip-flop enable. CLK is the clock input and represents the (posedge clk) in the always_ff block. D is the data input, in this case next_count. EN is the enable input or else if (enable) and SRST is the synchronous reset input or if (rst) section of our always_ff block. It appears that initially more complicated circuit designs are encapsulated and abstracted under PROC. It then appears that opt finds the simplest components to use when implementing our circuit design.

19 Report 18

Report 18

Include `techmap.png` in your report. The cell names now carry underscores and suffixes: $sdffe has become $_SDFFE_PP0P_ and the combinational cells have become $_XOR_, $_NOT_, $_AND_, and $_MUX_. These underscore-prefixed names are Yosys's technology-independent gate primitives, one step closer to real hardware. Notice that there are now two flip-flop instances rather than one — why? Observe also the labelled bubbles on the wires: `1:0` denotes a 2-bit wire (bits 1 and 0), `0:0` a single-bit wire (bit 0 only), and `1:1 -- 0:0` means bit 1 of the source connects to bit 0 of the destination. Why does Yosys slice the wires in this way? Finally, the suffix `PP0P` encodes the polarities of the flip-flop's ports.[a] Can you work out what each letter represents?

[a] At the time of writing, $_SDFFE_PP0P_ is documented in the Yosys manual.

Figure 24: Screenshot taken from Digital_Watch_Part_2.pdf

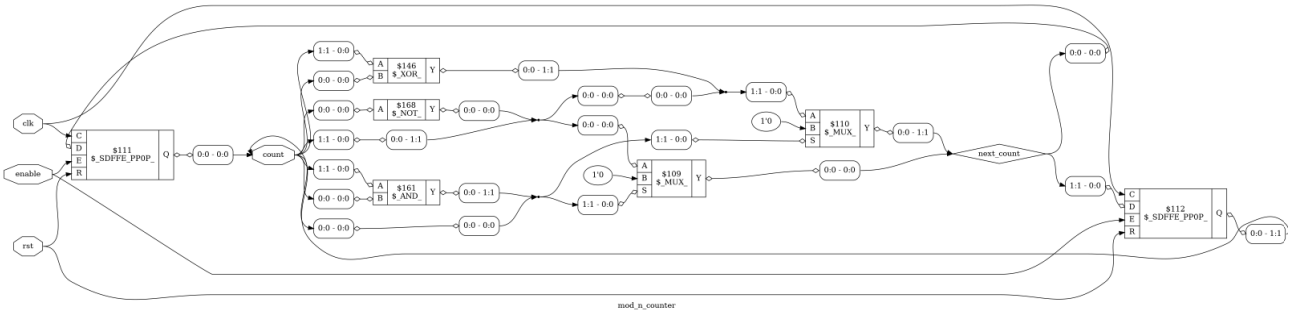


Figure 25: techmap.png generated for mod_n_counter.sv using Yosys

There are two flip flop instances as module instantiated a two-bit register this is then converted to a hardware primitive to become two-separate one bit flip-flops.

Yosys slices the wires this way as it needs to map any multibit signals to single bit output destination. Since this is representative of hardware primitive gates will require a single bit for its inputs.

PP0P represents a positive edge flip flop with a positive polarity enable and reset, with reset to zero.

20 Report 19

Report 19

Include `abc.png` in your report. Notice that the $_MUX_ present in every previous diagram has disappeared: `abc` replaced it with pure logic gates. The combinational logic has been reduced to just a $_NOT_ and a $_XOR_, feeding two $_SDFFE_PP0P_ flip-flops. From the diagram, draw the corresponding circuit: show the two flip-flop outputs (the two bits of `count`), the NOT gate, the XOR gate, and how they connect. Then, using truth tables, verify that this circuit correctly implements the counter's next-state logic for all four states.

Figure 26: Screenshot taken from Digital_Watch_Part_2.pdf

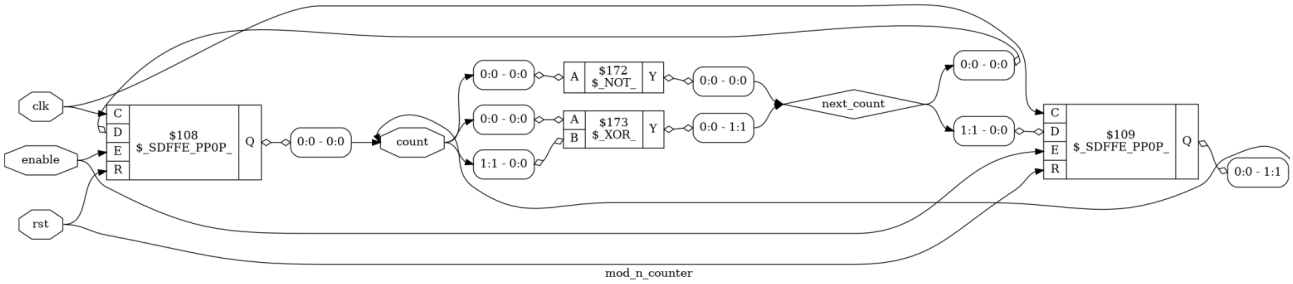


Figure 27: abc.png generated for mod_n_counter.sv using Yosys

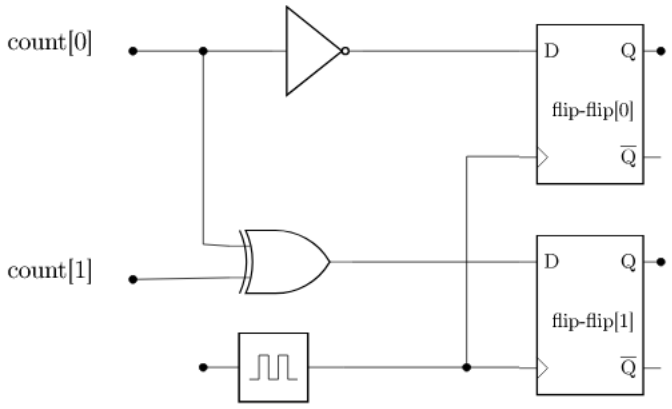


Figure 28: circuit_digram.png created at circuit2tikz.tf.fau.de/designer/

count[1]	count[0]	flip_flop[1]	flip_flop[0]
0	0	0	1
0	1	1	0
1	0	1	1
1	1	0	0

Figure 29: Truth table for circuit_diagram.png

The above truth table shows that our flip_flops store the values given by next_count which correctly shows out count incrementing and wrapping for a mod_n_counter.

21 Report 20

Report 20

If you add extra features, describe what you implemented, how you implemented it, and the reasoning behind any major design decisions.

Figure 30: Screenshot taken from Digital_Watch_Part_2.pdf

22 Report 21

Report 21

Based on your experience across both assignments, explain which parts of a digital watch are well suited to an FPGA and which would be easier to implement in software. Assume that any component responsible for counting or tracking time requires clock-cycle accuracy.

Figure 31: Screenshot taken from Digital_Watch_Part_2.pdf